L11-12 讲义：MCP 协议与 Skill 生态

课程: MBA 大模型智能体课程
课时: 第 11-12 讲 (共 3 小时)
适用: 讲师备课 / 学生预习与复习

本讲关键问题

作为 MBA 学生，学完这一讲你应该能回答：

1. **为什么 Agent 接入企业系统这么难？** → M×N 连接器爆炸问题
2. **MCP 是什么？为什么 Anthropic 要做这个标准？** → 协议价值与设计理念
3. **Skill/Plugin 生态有什么商业机会？** → 生态与商业模式
4. **我们公司应该自建还是用现成的？** → 选型决策框架
5. **安全风险是什么？如何防范？** → 工具调用安全

为什么这节课很重要

前面我们学了 Agent 的架构和工具调用。但在企业落地时，你会遇到一个巨大的问题：

场景：你想让 AI 助手能够：- 查询 CRM 系统的客户信息 - 读取 ERP 系统的库存数据 - 发送 Slack 消息通知团队 - 创建 Jira 工单跟踪任务 - 查询内部 Wiki 的知识

问题：- 每个系统的 API 都不一样 - 每个系统的认证方式都不一样 - 每次 Agent 升级，所有集成都要测试 - 每个系统升级 API，你的集成就可能挂掉

这就是"M×N 问题"：M 个 Agent × N 个系统 = M×N 条需要维护的集成链路

MCP（Model Context Protocol）的价值：把 M×N 问题变成 M+N 问题，通过标准协议让 Agent 和工具"说同一种语言"。

目录

1. 工具集成痛点
2. MCP 协议原理
3. JSON-RPC 架构详解
4. Claude Desktop 配置实战
5. Agent-Computer Interface 设计
6. Skill 系统与组织复用
7. 生态现状与选型策略
8. 参考资料

---

1. 工具集成痛点

1.1 M×N 连接器爆炸—企业最头疼的问题

真实案例：某中型企业想部署 AI 助手

现状：- 3 个不同的 Agent 产品（客服、销售、运营）- 20 个内部系统（CRM、ERP、OA、Slack、Jira…）- **结果**：需要维护 3 × 20 = 60 条集成链路

每条链路都要处理：- 认证（OAuth、API Key、SSO…）- 限流（每个系统限流规则不同）- 错误处理（超时、重试、降级）- 版本兼容（API 升级怎么办?）

成本：- 初期开发：每条链路平均 2 人周 = 120 人周 = **约 3 人年** - 持续维护：每年约 0.5 人年 - 任一系统 API 变更，可能影响多条链路

更好的方案：标准化

方案	接入复杂度	维护复杂度	人力投入
各做各的	M×N	极高	3 人年 +
标准协议	M+N	可控	0.5 人年

1.2 四大核心痛点（MBA 视角）

痛点	技术表现	商业影响
重复开发	每个 Agent 都要写同样的集成	人力浪费
维护噩梦	改一处要测试全部	迭代缓慢
升级风险	底层 API 变更影响全局	系统脆弱
安全漏洞	每条链路都是攻击面	合规风险
重复造轮子	每个团队做"同类连接器"，能力定义不统一	成本上升，知识无法沉淀
稳定性不可预期	API 限流、字段变化、权限过期导致失败	自动化链路无法承诺 SLA
安全与合规难治理	谁可以调用"删除客户数据"？审计日志在哪?	安全策略散落在脚本里
组织协作成本高	平台/业务/安全团队语言不一致	"能跑起来"和"可运营"有巨大鸿沟

结论：需要像 HTTP 一样的"工具调用标准层"。

---

2. MCP 协议原理

2.1 MCP 是什么

> **MCP（Model Context Protocol）是面向模型应用的上下文与工具交互标准。**

- 由 Anthropic 于 2024 年 11 月发布的开放协议
- 对标思路：HTTP 统一 Web 通信，SQL 统一数据库查询，MCP 统一"模型 ↔ 外部能力"

2.2 MCP 的设计哲学

1. **协议优先**：先统一接口，再谈生态繁荣
2. **能力可发现**：工具能力应可枚举、可理解
3. **模型无关**：不绑定某个 LLM 厂商
4. **传输可替换**：stdio 本地、HTTP 远程
5. **安全可治理**：权限、审计、边界明确

2.3 三角色：Host / Client / Server

角色	典型实例	职责
Host	Claude Desktop, IDE Agent	管理会话与策略
Client	Host 中的 MCP 客户端	发送 JSON-RPC 请求

角色	典型实例	职责
Server	GitHub/DB/Filesystem 服务	暴露标准能力

```
User


Host (Claude Desktop / IDE)
    MCP Client A   JSON-RPC   MCP Server: github
    MCP Client B   JSON-RPC   MCP Server: postgres
    MCP Client C   JSON-RPC   MCP Server: filesystem
```

2.4 四大能力模型

能力	作用	典型场景
Tools	可执行动作	创建 Issue、发消息、写库
Resources	可读取上下文	文件、记录、配置
Prompts	模板化提示	评审模板、任务模板
Sampling	Server 请求模型推理	复杂链式决策（进阶）

Tools 示例

```json
{
  "name": "create_issue",
  "description": "Create a GitHub issue",
  "inputSchema": {
    "type": "object",
    "properties": {
      "repo": {"type": "string"},
      "title": {"type": "string"}
    },
    "required": ["repo", "title"]
  }
}
```

Resources 示例

```json
{
  "uri": "postgres://sales/orders?date=2026-02-01",
  "name": "Orders Snapshot",
  "mimeType": "application/json"
}
```

2.5 MCP 生命周期

1. 建立连接（stdio/HTTP）
2. `initialize` 协商能力与版本
3. `initialized` 通知完成
4. `tools/list` / `resources/list` 能力发现
5. `tools/call` 执行与返回
6. 会话结束与资源回收

---

3. JSON-RPC 架构详解

3.1 JSON-RPC 2.0 基础

MCP 使用 JSON-RPC 2.0 作为消息骨架：

- 核心字段：`jsonrpc`、`id`、`method`、`params`
- 响应字段：`result` 或 `error`
- 通知消息：无 `id`

3.2 完整消息示例

Request

```
{
  "jsonrpc": "2.0",
  "id": "req-42",
  "method": "tools/call",
  "params": {
    "name": "create_issue",
    "arguments": {
      "repo": "org/project",
      "title": "MCP timeout on prod"
    }
  }
}
```

Response (成功)

```
{
  "jsonrpc": "2.0",
  "id": "req-42",
  "result": {
    "content": [{"type": "text", "text": "Issue #128 created"}]
  }
}
```

Response (错误)

```
{
  "jsonrpc": "2.0",
  "id": "req-42",
  "error": {
    "code": -32001,
    "message": "Permission denied",
    "data": {
      "policy": "github.write.prod",
      "hint": "Request temporary approval"
    }
  }
}
```

3.3 错误处理四层模型

错误层	示例	处理建议
协议错误	非法 JSON-RPC	直接拒绝，记录日志
参数错误	缺字段/类型错	返回可读错误 + 修复建议

错误层	示例	处理建议
业务错误	权限不足/资源不存在	提示用户与策略系统
系统错误	网络超时/下游异常	重试 + 降级 + 告警

3.4 传输层：stdio vs HTTP

维度	stdio	Streamable HTTP
部署	本地进程	远程服务
门槛	低	中高
安全	本机边界为主	需鉴权/网关
适用	个人工具、本地研发	团队共享、生产环境

---

4. Claude Desktop 配置实战

4.1 配置文件位置

平台	路径
macOS	`~/Library/Application Support/Claude/claude_desktop_config.json`
Windows	`%APPDATA%\Claude\claude_desktop_config.json`

4.2 配置结构示例

单 Server 配置

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/Users/you/Documents"],
      "env": {}
    }
  }
}
```

多 Server 配置

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/Users/you/work"]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
    }
  }
}
```

4.3 常见排障清单

1. 命令不存在（`command not found`）
2. 包安装失败（网络或权限）
3. 环境变量未注入（token 为空）
4. 路径权限不足（文件系统拒绝）
5. Server 启动后立即退出（参数错误）

定位方法：- 先看 Claude Desktop 日志 - 在终端独立运行 Server 确认是否可启动 - 逐步最小化配置：先 filesystem，再加复杂 Server

---

5. Agent-Computer Interface 设计

- ☐ 来源: Anthropic - Building Effective Agents

5.1 ACI 设计的重要性

核心观点：ACI (Agent-Computer Interface) 和 HCI 一样重要

```
人类用户  GUI → 计算机系统
AI Agent  ACI → 计算机系统
```

5.2 工具设计原则

原则	说明	实践
站在模型角度思考	描述是否足够清晰?	用 Workbench 测试模型如何理解
Poka-yoke（防呆）	让错误难以发生	参数校验、枚举值限制
给足思考空间	避免让模型"写进死角"	提供回退选项
测试失败模式	观察模型如何失败	收集真实 case 优化描述

5.3 好 Schema vs 坏 Schema

维度	好 Schema	坏 Schema
命名	`customer_id`	`id`
枚举	`priority: low/med/high`	`level: 1/2/3`
描述	业务上下文完整	只有技术术语
校验	类型/范围明确	运行时才报错

5.4 CLAUDE.md 项目级指令

在项目根目录放置 `CLAUDE.md` 文件：

```
# CLAUDE.md

## 项目背景
这是一个电商后台管理系统...

## 代码规范
- 使用 TypeScript strict mode
- 组件使用函数式写法
- 测试覆盖率 > 80%
```

```
## 常见陷阱
- 不要直接修改 config/prod.json
- API 变更需要同步更新 mock
```

作用：每次会话开始自动加载，确保 Agent 了解项目上下文

5.5 工具格式选择

格式	优点	缺点
Diff	紧凑	需要预知行数
全文重写	简单	token 消耗大
Markdown 代码块	自然	无需额外转义
JSON 内嵌代码	结构化	需转义换行引号

建议：选择模型"在互联网上见过最多"的格式

---

6. Skill 系统与组织复用

6.1 什么是 Skill

> **Skill = 让 Agent 通过"说明 + 脚本 + 资源"获得稳定能力。**

- 常见组成：`SKILL.md`、脚本、配置、模板
- Agent 先读说明，再决定调用什么命令
- 特别适合内部流程与垂直场景

6.2 Skill vs MCP

维度	MCP Server	Skill
标准化	强（跨 Host）	弱（平台相关）
上手速度	中	快
复用范围	广	团队内高
治理复杂度	高	中

定位：Skill 负责快速试错，MCP 负责标准化沉淀

6.3 Skill 目录结构

```
my-skill/
  SKILL.md          # 核心说明：何时用、怎么用
  scripts/
    run.sh
    validate.py
  templates/
    report.md
  config.yaml       # 环境/参数配置
```

6.4 SKILL.md 应写什么

```
## 命令
python scripts/generate_report.py --date {date} --team {team}

## 输入约束
- date: YYYY-MM-DD
- team: sales | ops | finance

## 失败处理
- 若数据库连接失败，返回"retry_after=60"
```

原则：让"新同事 5 分钟可上手"

6.5 Skill 质量检查清单

- ☐ 是否有明确"适用/不适用场景"
- ☐ 是否提供最小可运行示例
- ☐ 错误是否可定位、可恢复
- ☐ 是否声明了权限与风险
- ☐ 是否有版本号与变更记录

---

7. 生态现状与选型策略

7.1 2026 MCP 生态观察

1. 工具数量增长很快，但质量参差
2. 通用 Server 成熟更快（GitHub/DB/Filesystem）
3. 企业更关心：安全、稳定、审计
4. 生态正在从"玩具化"走向"生产化"

7.2 生态能力地图

类别	代表 Server	典型价值
开发协作	GitHub/GitLab	工程自动化
数据分析	Postgres/BigQuery	报表与决策
办公协同	Slack/Notion	沟通与知识流
本地执行	Filesystem/Shell	自动化运维

7.3 选型矩阵

业务条件	推荐
跨多 AI 平台复用	MCP 优先
内部流程快速试错	Skill 优先
强合规与审计要求	MCP + 网关治理
既要速度又要复用	混合架构

7.4 0-1-10 落地路线图

阶段	时间	目标
0 到 1	2 周	1 个关键流程打通
1 到 N	1-2 月	沉淀标准 Tool Schema 与日志规范
10+	季度	平台化治理、权限自动化、成本看板

7.5 安全分层建议

1. **身份层**：OAuth / API Key / 短期令牌
2. **权限层**：按工具、按参数、按环境授权
3. **执行层**：沙箱、命令白名单、只读默认
4. **审计层**：谁在何时调用了什么，结果如何

7.6 可观测性指标

- 工具调用成功率（按工具维度）
- P95 响应时间 / 超时率
- 参数错误率（提示 Schema 质量）
- 安全拒绝率（权限策略是否合理）
- 单次任务工具调用次数（成本与稳定性信号）

---

8. 参考资料

官方文档

- MCP Official Documentation
- MCP Servers Repository
- MCP Inspector
- JSON-RPC 2.0 Specification
- Claude Desktop

最佳实践 □

- Anthropic: Building Effective Agents - ACI 设计
- Anthropic: CLAUDE.md Best Practices
- Simon Willison: LLM Tools

平台与生态

- ClawHub - Skill 市场
- Smithery - MCP 生态目录
- Coze - 国内 Agent 平台
- Dify - 流程编排平台

实践作业

作业 1：MCP 配置实战

1. 安装 Claude Desktop
2. 配置 filesystem Server
3. 用 MCP Inspector 观察一次完整的 `tools/list` 和 `tools/call` 消息

作业 2：设计一个 Skill

创建一个"日报生成 Skill"，包含：- SKILL.md 说明文档 - 输入参数定义 - 执行脚本 - 失败处理策略

作业 3：企业集成方案

为你的公司设计 MCP/Skill 混合方案：1. 列出需要接入的 3-5 个内部系统 2. 判断每个用 MCP 还是 Skill 3. 设计权限与审计方案 4. 绘制部署架构图

管理者要点总结

关注点	核心信息
业务价值	标准化后，跨部门自动化项目交付周期可从"按月"缩短到"按周"
主要风险	无统一协议会产生"集成债务"；凭证管理不当可能导致数据暴露
预算建议	前期协议改造投入约占 PoC 预算 10%-20%；安全与权限治理预算不低于 15%
选型原则	Skill 负责快速试错，MCP 负责标准化沉淀，组合可兼顾速度与规模

本讲义最后更新：2026-02-28